

Bibliothèques Python — Référence Zeta-Lab

Version enrichie — Projet Riemann_Lab · hprzeta Mise à jour : 2 juin 2026 · Auteur : hprzeta

Environnement

```
# Activation obligatoire avant tout script
source ~/projet_zeta/zeta_env/bin/activate
export PYTHONPATH="${PYTHONPATH}:${HOME}/projet_zeta/src"
python mon_script.py
```

Modules du projet (dans ~/projet_zeta/src/calculs/optimisation/) :

```
theta_rapide.py          #  $\theta(t)$  asymptotique + Z_fast
riemann_siegel_batch.py # Z_batch GPU/CPU, scanner_batch + Illinois
turing_validation.py     #  $N(T)$ ,  $S(T)$ , valider_turing
parallel_scanner.py      # calcul multiprocessing
compute_zeros_v3.py      # orchestrateur principal
```

1. mpmath — Arithmétique haute précision

Rôle : calcul exact de $\zeta(s)$, $\theta(t)$, $\Gamma(s)$, affinage Illinois.

```
from mpmath import mp, mpc, mpf, zeta, loggamma, zetazero
from mpmath import siegelz, siegeltheta # wrappers haut niveau
from mpmath import findroot, arg, re, im, pi, log

# — Précision adaptative (règle v3) —————
mp.dps = 25 # détection rapide (changement de signe Z)
mp.dps = 35 # affinage Illinois (atteint tol=1e-12)
mp.dps = 50 # validation / publication des 1000 premiers zéros

# —  $\theta(t)$  exact (pour  $t < 20$  ou vérification) —————
theta_exact = float(
    mp.im(loggamma(mpf("0.25") + mpc(0, t) / 2))
    - (t / 2) * mp.log(mp.pi)
)

# —  $Z(t)$  optimisé — un seul appel à zeta() —————
import math
s = mpc("0.5", t)
z = zeta(s) # appel unique
val = math.cos(th) * float(z.real) - math.sin(th) * float(z.imag)

# — Zéros via zetazero (lent, pour vérification seulement) —————
```

```

zetazero(1)    # 0.5 + 14.1347...j  - précis mais ~1 s/zéro
zetazero(10)   # 0.5 + 49.7738...j

# — Affinage Illinois —————
mp.dps = 35
t0 = findroot(
    lambda x: Z_fast(float(x), dps=35),
    (t_a, t_b),          # intervalle avec changement de signe
    solver="illinois",
    tol=1e-12,
    maxsteps=80,
)
# (!) tol=1e-12 cohérent avec 35 dps.  tol=1e-20 à 35 dps → IMPOSSIBLE (bug v2)

# — Sauvegarde/restauration de précision —————
dps_save = mp.dps
mp.dps = 35
# ... calcul ...
mp.dps = dps_save  # toujours restaurer après usage local

```

2. numpy — Calcul vectorisé (CPU)

Rôle : $\theta(t)$ vectorisé, matrice de phases RS, détection de signes.

```

import numpy as np

# —  $\theta(t)$  vectorisé (asymptotique de Stirling) —————
LOG_2PI = math.log(2 * math.pi)
PI_8     = math.pi / 8.0

t2 = ts * ts;  t3 = t2 * ts;  t5 = t3 * t2
thetas = (
    (ts / 2.0) * (np.log(ts) - LOG_2PI)
    - ts / 2.0 - PI_8
    + 1.0 / (48.0 * ts)
    + 7.0 / (5760.0 * t3)
    - 31.0 / (80640.0 * t5)
)

# — Matrice de phases RS (œcur de Z_batch) —————
ns      = np.arange(1, N_max + 1, dtype=np.float64)
log_ns  = np.log(ns)
inv_sqn = 1.0 / np.sqrt(ns)

phases = thetas[:, None] - ts[:, None] * log_ns[None, :]  # shape (M, N_max)
Z       = 2.0 * np.dot(np.cos(phases), inv_sqn)           # shape (M,)

# — Détection des changements de signe —————
signes  = np.sign(Z_vals)
passages = np.where(np.diff(signes) != 0)[0]  # indices des changements

# — Espacements normalisés (conjecture de Montgomery) —————
ecarts  = np.diff(zeros)
t_mid   = zeros[:-1]
delta_n = ecarts * np.log(np.array(t_mid) / (2 * math.pi)) / (2 * math.pi)

```

3. CuPy — Calcul vectorisé GPU (NVIDIA GTX 960M)

Rôle : accélération $\times 8$ à $\times 12$ de Z_{batch} via CUDA.

```
import cupy as cp

# — Détection GPU —————
n_devices = cp.cuda.runtime.getDeviceCount() # 0 si GPU inactive
# (!) Activer le GPU : sudo prime-select nvidia puis redémarrage
# (!) Paquets : pip install cupy-cudal2x (doit matcher CUDA 12.2)

# — Calcul GPU identique à numpy, backend différent —————
ts_g = cp.asarray(ts_cpu, dtype=cp.float64)
thetas_g = cp.asarray(thetas_cpu, dtype=cp.float64)
log_ns_g = cp.asarray(log_ns, dtype=cp.float64)
inv_sqn_g = cp.asarray(inv_sqn, dtype=cp.float64)

phases = thetas_g[:, None] - ts_g[:, None] * log_ns_g[None, :]
Z_g = 2.0 * cp.dot(cp.cos(phases), inv_sqn_g)

Z_cpu = cp.asnumpy(Z_g) # retour vers numpy

# — Limites mémoire GTX 960M (4 GB VRAM) —————
# bloc recommandé : int(1_500_000_000 / (N_max * 8))
# T = 10 000 (N_max=39) → bloc=500 000 → 156 MB $|checkmark$
# T = 100 000 (N_max=126) → bloc=200 000 → 201 MB $|checkmark$

# (!) BLAS thread contention : benchmarker les modes isolément (pas en concurrent)
```

4. multiprocessing — Parallélisation (4 cœurs)

Rôle : partitionner $[T_{\min}, T_{\max}]$ sur N workers. $\times 4$ de gain.

```
import multiprocessing

# (!) Ne PAS utiliser joblib avec mpmath :
# GMP (MPFR) a un état global non thread-safe → corruption mémoire.
# Solution : multiprocessing (fork → chaque processus a sa propre copie GMP)

n_workers = multiprocessing.cpu_count() # 4 sur i7

# — Partitionnement —————
longueur = (T_MAX - T_MIN) / n_workers
segments = [
    (T_MIN + i * longueur, T_MIN + (i + 1) * longueur)
    for i in range(n_workers)
]

# (!) Chevauchement de 2-STEP aux jonctions pour ne rater aucun zéro de bord

# — Lancement —————
with multiprocessing.Pool(n_workers) as pool:
    resultats = pool.map(worker_func, args_list)

# — Fusion et déduplication —————
zeros_bruts = [t for segment in resultats for t in segment]
zeros_bruts.sort()
zeros = dedupliquer(zeros_bruts, tolerance=0.001)
```

5. pandas — Sauvegarde CSV

Rôle : stocker les zéros calculés avec métadonnées.

```
import pandas as pd
from pathlib import Path
from datetime import datetime

horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")
nom_csv = f"zeros_v3_T{T_MAX:.0f}_{horodatage}.csv"

df = pd.DataFrame({
    "n": range(1, len(zeros) + 1),
    "partie_imaginaire": zeros,
    "T_MAX": T_MAX,
    "methode": "RS-detection_+Illinois-affinage",
    "step_adaptatif": STEP,
    "n_workers": N_WORKERS,
    "turing_complet": True,
    "calcule_le": horodatage,
})
df.to_csv(str(Path("calculs") / nom_csv), index=False)

# — Chargement ultérieur —————
df = pd.read_csv("zeros_zeta_T10000_20260424_205325.csv")
zeros = df["partie_imaginaire"].tolist()
```

6. matplotlib — Visualisation (sans blocage)

Rôle : tracer $Z(t)$, espacements, droite critique.

```
import matplotlib.pyplot as plt
import numpy as np

# (!) TOUJOURS plt.savefig() + plt.close() – JAMAIS plt.show() en production
# plt.show() est BLOQUANT : empêche la génération du log/CSV après le graphique.

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle(f"Zéros_{len(zeros)}_zéros[T={T_MAX:.0f}]")

# — Z(t) avec marqueurs aux zéros —————
ax = axes[0]
ax.plot(t_vals, Z_vals, 'b-', linewidth=0.8, label='Z(t)')
ax.axhline(0, color='k', linewidth=0.5)
for t0 in zeros:
    ax.axvline(t0, color='r', linewidth=0.5, alpha=0.4)

# — Histogramme espacements vs GUE —————
ax = axes[1]
ax.hist(delta_n, bins=50, density=True, alpha=0.75, color='steelblue')
s_vals = np.linspace(0, 4, 200)
gue = (math.pi / 2) * s_vals * np.exp(-math.pi * s_vals**2 / 4)
ax.plot(s_vals, gue, 'r-', linewidth=2, label='GUE(Wigner-Dyson)')

# — Droite critique —————
ax = axes[2]
ax.scatter([0.5] * len(zeros), zeros, s=3, color='darkblue', alpha=0.4)
ax.axvline(0.5, color='r', linestyle='--', linewidth=1.5)
```

```
plt.tight_layout()
plt.savefig("zeros_v3.png", dpi=150)
plt.close()      # ← libère la mémoire, ne bloque pas
```

7. loguru — Logging structuré

Rôle : journal d'exécution persistant avec rotation.

```
from loguru import logger
from pathlib import Path

logger.add(
    Path("~/projet_zeta/logs/projet_zeta.log"),
    rotation="100MB",
    retention="30days",
    format="{time:YYYY-MM-DD_HH:mm:ss} | {level} | {message}"
)

logger.info(f"Calcul_v3_démarré: T_MAX={T_MAX:.0f}")
logger.success(f"Zéro_{n:5d}: t={t:.14f} (écart_LMFDB: {ecart:.2e})")
logger.warning(f"Worker_{wid}: surplus_de_{delta} zéros aux bords")
logger.error(f"Illinois non convergé: [{t_a:.4f}, {t_b:.4f}]")
```

8. tqdm — Barres de progression

```
from tqdm import tqdm

for t in tqdm(np.arange(T_MIN, T_MAX, STEP), desc="Scan_Z(t)", unit="pt"):
    # calcul...
```

9. sympy — Calcul symbolique (exploration, pas production)

```
from sympy import zeta, gamma, pi, I, re, im, Symbol, simplify
from sympy.ntheory import primerange, isprime, nextprime

s = Symbol('s')
zeta(s)                # objet symbolique
list(primerange(1, 100)) # premiers jusqu'à 100
nextprime(100)         # 101
```

Note : sympy n'est pas utilisé dans le pipeline de production (trop lent).
Utiliser mpmath pour les calculs numériques de haute précision.

10. psutil — Monitoring système

```
import psutil

n_cores = psutil.cpu_count(logical=True)      # 4 (i7)
ram_gb   = psutil.virtual_memory().total / 1e9 # 8.0 GB
ram_pc   = psutil.virtual_memory().percent     # % utilisé
```

11. Vérification GPU avant lancement

```
# Vérifier que le GPU NVIDIA est actif
nvidia-smi

# Si "no devices found" :
sudo prime-select nvidia
# → reboot nécessaire

# Paquet CuPy correct pour CUDA 12.2 :
pip install cupy-cuda12x --break-system-packages
```

12. Arb / FLINT — affinage rapide haute précision (levier Phase C)

Pourquoi cette section : la §17 de Formules_zeta.md (mur de latence) montre que le goulot du calcul est la **latence d'un appel** `mpmath.siegelz` (~ 296 ms à $t \approx 9000$), répétée ~ 27 fois par zéro. Le **seul levier réaliste** pour des positions $< 10^{-10}$ sous 30 min n'est ni le GPU ni la RAM (absents de la formule de coût), mais le remplacement de `mpmath` par **Arb** à l'étape d'affinage.

Rôle : évaluer $Z(t)$ de Hardy en arithmétique de boules (intervalles certifiés, en C), $\approx \times 10\text{--}20$ plus vite que `mpmath.siegelz` à précision égale.

```
# — Installation (Ubuntu) —————
sudo apt install libflint-dev libarb-dev          # bibliothèques C
pip install python-flint --break-system-packages  # binding Python (FLINT + Arb)
```

```
# — Précision en BITS  $\approx (dps \cdot 3.33)$  —————
from flint import arb, ctx
ctx.prec = 200      # ~60 chiffres décimaux

# Fonction clé côté Arb : acb_dirichlet_hardy_z →  $Z(t)$  de Hardy avec bornes
# d'erreur rigoureuses. Si python-flint ne l'expose pas directement, l'appeler
# via un petit wrapper ctypes vers libarb (acb_dirichlet_hardy_z).
```

(!) Le facteur $\times 10\text{--}20$ est une **estimation à mesurer** : benchmark `affinage_arb.py` contre `siegelz` sur $[300, 700]$ AVANT toute réécriture. Bonus à grand volume : l'algorithme **Odlyzko-Schönhage** (multi-évaluation par FFT) casse le coût asymptotique — pertinent seulement pour $T \gg 10\,000$.

Outil	Rôle	Statut projet
mpmath.siegelz	affinage actuel (Voie B)	production, lent à grand t
Arb acb_dirichlet_hardy_z	affinage cible (levier §17)	à benchmarker
FLINT	dépendance d'Arb (entiers, polynômes)	requis pour Arb

13. Tableau récapitulatif — rôle par bibliothèque

Bibliothèque	Usage dans v3	Mode
mpmath	$\theta(t)$ exact, Z_{fast} , affinage Illinois, validation	CPU précis
numpy	$\theta(t)$ vectorisé, phases RS, détection signes	CPU rapide
cupy	Z_{batch} GPU (si GTX 960M active)	GPU $\times 10$
multiprocessing	Parallélisation sur 4 cœurs	CPU $\times 4$
pandas	Sauvegarde/chargement CSV zéros	I/O
matplotlib	$Z(t)$, espacements, droite critique	Visualisation
loguru	Journal d'exécution complet	Logging
tqdm	Barres de progression	Interface
sympy	Exploration symbolique seulement	Hors prod.
psutil	Monitoring CPU/RAM	Diagnostic
arb / python-flint	Affinage $Z(t)$ rapide (levier §17, à benchmarker)	CPU certifié

Auteur : hprzeta · Dernière mise à jour : 2 juin 2026 — 390 lignes

§13 — workprec vs fp : précision locale en mpmath (3 juin 2026)

Contexte

Découvert lors du diagnostic pipeline v4.1 (3 juin 2026) : workprec ne contrôle pas la précision des fonctions mpmath internes.

workprec — précision de la boucle, pas de la fonction

```
import mpmath as mp
mp.mp.dps = 35    # global : 35 dps

with mp.workprec(50):    # 50 bits ≈ 15 dps – contexte local
    x = mp.siegelz(100.0)    # (!) siegelz re-lit mp.dps = 35 → précision 35 dps
    y = mp.findroot(
        mp.siegelz, (14.1, 14.2),
        solver="illinois"
    )
    # La boucle Illinois tourne à 50 bits, MAIS chaque éval siegelz = 35 dps
```

Usage correct de workprec : opérations arithmétiques pures (additions, multiplications, racines) — PAS pour les fonctions de haut niveau comme siegelz, zeta, gamma.

fp — float64 natif (×40 plus rapide que dps=35)

```
# float64 natif — ~1e-15 de précision, pas de surcharge multi-précision
z = mp.fp.siegelz(t)          # float64
theta = mp.fp.siegeltheta(t)

# findroot en float64 (bracket obligatoire pour stabilité) :
zero = mp.fp.findroot(
    mp.fp.siegelz, (a, b),
    solver="illinois", tol=1e-12, maxsteps=80,
)
```

Quand utiliser fp : $t < 300$ ($N < 7$ termes RS), $\text{tol} = 1e-12$, bracket fourni. float64 ($\varepsilon \approx 10^{-16}$) est sous la tolérance → précision suffisante.

Quand NE PAS utiliser fp : grand t , précision $< 1e-12$ exigée, ou validation LMFDB (qui nécessite ≥ 30 dps pour comparer γ à 14 décimales significatives).

Résumé comparatif

API	Précision	Coût relatif	Usage recommandé
mp.siegelz(t) dps=35	~1e-35	référence (1×)	affinage final, LMFDB
mp.siegelz(t) dps=15	~1e-15	~×4 plus rapide	— (workprec ne marche pas)
mp.fp.siegelz(t)	~1e-15	~×40 plus rapide	$t < 300$, bracket fourni
C/libmpfr PREC=170 bits	~1e-50	~×10 vs mpmath	$t \geq 300$, Illinois_C

Auteur : hprzeta — Riemann_Lab — Mise à jour : 3 juin 2026